People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

University of Biskra — Department of Computer Science

Repair Manager

MERN Stack Application

Auto Repair Shop Management System

Technical Report — Academic Year 2024/2025

Prepared By

Noui AbdAllah **Oussama Charaf Eddine**
Sebaa **MEGUEDDEM Yacine**

Under the Supervision of
Dr. HIOUANI Rima

April 30, 2026

Contents

Abstract **3**

1 Introduction **4**
1.1 Context and Motivation 4
1.2 Project Objectives 4
1.3 Report Structure 5

2 Technology Stack Overview **6**
2.1 The MERN Stack 6
2.1.1 MongoDB 6
2.1.2 Express.js 6
2.1.3 React 7
2.1.4 Node.js 7
2.2 Supporting Libraries 7

3 System Architecture **8**
3.1 High-Level Architecture 8
3.2 Request Flow 8
3.3 Project Directory Structure 8

4 Data Modeling **10**
4.1 Database Design 10
4.2 User Model 10
4.3 RepairJob Model 11
4.3.1 Tracking Code Generation 11

5 API Design **13**
5.1 RESTful Architecture 13
5.2 Authentication Endpoints 13
5.3 Repair Job Endpoints 14
5.4 Query Filtering 14

6 Frontend Architecture **15**
6.1 Application Structure 15
6.2 Core Components 15
6.3 Navigation and Tabs 15
6.4 State Management 16
6.5 Dashboard Metrics 16

7 Security Implementation **18**
7.1 JWT Authentication 18
7.2 Password Security 19
7.3 CORS Policy 19
7.4 User Data Isolation 19

8 Deployment **20**
8.1 Overview 20
8.2 Backend Deployment (Render) 20
8.3 Frontend Deployment (Vercel) 20
8.4 Health Check 21

9 Features Summary **22**
9.1 Implemented Features 22

10 Conclusion and Perspectives **23**
10.1 Conclusion 23
10.2 Future Enhancements 23
10.3 Acknowledgements 24

Abstract

This report presents the design, implementation, and deployment of **Repair Manager**, a full-stack web application built using the **MERN** stack (MongoDB, Express.js, React, Node.js). The system is designed for auto repair shop garages to manage repair jobs efficiently with role-based access, real-time status tracking, and a customer-facing tracking portal.

The application addresses a core business need in the automotive maintenance sector: streamlining job intake, assignment, status updates, cost estimation, and completion archiving. It implements industry-standard security practices including **JWT-based authentication**, bcrypt password hashing, CORS protection, and input validation.

The project was deployed successfully using **Render** (backend) and **Vercel** (frontend), with a live cloud-hosted MongoDB Atlas database. The report covers system architecture, data modeling, API design, frontend component structure, security implementation, and deployment procedures.

Keywords: MERN Stack, MongoDB, Express.js, React, Node.js, REST API, JWT Authentication, Auto Repair Management, Web Application.

Chapter 1

Introduction

1.1 Context and Motivation

The management of an auto repair shop involves complex workflows: tracking customer vehicles, assigning repair tasks, monitoring job status, estimating and recording costs, and maintaining historical records. Traditional paper-based or spreadsheet approaches are error-prone, not scalable, and offer no real-time visibility to stakeholders.

Repair Manager was conceived as a modern, web-based solution to address these challenges. By leveraging a full-stack JavaScript ecosystem, the team was able to build both a powerful backend API and a responsive user interface using a unified language and tool chain.

1.2 Project Objectives

The project aims to:

- Provide a secure, multi-user platform for garage administrators.
- Enable full CRUD operations on repair job records.
- Generate unique tracking codes for each repair job.
- Allow customers to look up their repair status without logging in.
- Offer a real-time dashboard with statistics and financial summaries.
- Deploy the system to a publicly accessible URL using cloud platforms.

1.3 Report Structure

This report is organized as follows: Chapter 2 reviews related technologies and the MERN stack. Chapter 3 presents the system architecture. Chapter 4 covers data modeling. Chapter 5 documents the API. Chapter 6 describes the frontend. Chapter 7 discusses security. Chapter 8 covers deployment. Chapter 9 concludes the report.

Chapter 2

Technology Stack Overview

2.1 The MERN Stack

The **MERN** stack is a popular JavaScript-based framework for building full-stack web applications. It consists of four core technologies:

Letter	Technology	Version
M	MongoDB — NoSQL document database	Atlas / 7.x
E	Express.js — Web framework for Node.js	4.x
R	React — Frontend JavaScript library	18.x
N	Node.js — JavaScript runtime environment	18.x

Table 2.1: The MERN Stack Components

2.1.1 MongoDB

MongoDB is a NoSQL database that stores data as flexible JSON-like documents (BSON). It does not require a fixed schema, making it well-suited for rapidly evolving applications. In this project, MongoDB Atlas — a cloud-hosted database service — was used in production.

2.1.2 Express.js

Express.js is a minimal, unopinionated web framework for Node.js. It simplifies the creation of RESTful APIs through a clean routing system and middleware pipeline. This project uses Express to define and protect all API endpoints.

2.1.3 React

React is a component-based frontend library developed by Meta. It enables building interactive UIs through composable components and a virtual DOM for efficient rendering. The frontend is built as a single-page application (SPA) with React 18.

2.1.4 Node.js

Node.js is an asynchronous, event-driven JavaScript runtime. It allows JavaScript to run on the server side, enabling the backend and frontend to share a common language and tooling ecosystem.

2.2 Supporting Libraries

Library	**Purpose**
Mongoose	ODM (Object-Document Mapper) for MongoDB schema definition
JSON Web Token (jsonwebtoken)	Stateless authentication token generation and verification
bcryptjs	Secure password hashing
dotenv	Environment variable management
cors	Cross-Origin Resource Sharing middleware
Axios	HTTP client for API requests from the frontend
Tailwind CSS	Utility-first CSS framework for rapid UI styling

Table 2.2: Supporting Libraries

Chapter 3

System Architecture

3.1 High-Level Architecture

The system follows a classical three-tier architecture:

Three-Tier Architecture

1. **Presentation Layer** — React single-page application, served via Vercel CDN
2. **Application Layer** — Node.js / Express REST API, hosted on Render
3. **Data Layer** — MongoDB Atlas cloud database

3.2 Request Flow

The typical request lifecycle is as follows:

1. The user interacts with the React frontend (e.g., submits a repair form).
2. Axios sends an HTTP request to the Express API with a `Bearer` JWT token in the `Authorization` header.
3. The `authMiddleware` verifies the token; if invalid, a `401 Unauthorized` response is returned.
4. The route handler queries or mutates MongoDB via Mongoose.
5. The API returns a JSON response, which React renders to the user.

3.3 Project Directory Structure

```
MERNPROJECT/
          backend/
                middleware/
                      authMiddleware.js     # JWT verification
                models/
                      RepairJob.js          # Repair job schema
                      User.js               # User schema
                routes/
                      authRoutes.js         # /api/auth endpoints
                      repairRoutes.js       # /api/repairs endpoints
                server.js                 # Entry point
                .env                      # Environment variables

          frontend/
              src/
                    components/
                          AuthForm.jsx       # Login / Signup UI
                          CustomerTrack.jsx # Public tracking portal
                          RepairForm.jsx     # Add / Edit repair form
                          RepairItem.jsx     # Single repair card
                          RepairList.jsx     # Repair jobs list
                    App.jsx                # Root component + state
                    apiConfig.js           # API base URLs
                    styles.css             # Global styles
              .env                       # Frontend environment
```

Listing 3.1: Project Structure

Chapter 4

Data Modeling

4.1 Database Design

The application uses two primary MongoDB collections, defined through Mongoose schemas.

4.2 User Model

The `User` schema manages garage administrator accounts.

```
const UserSchema = new mongoose.Schema({
  name:     { type: String, required: true, minlength: 2 },
  email:    { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6, select: false },
}, { timestamps: true });

// Pre-save hook: hash password with bcrypt
UserSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});
```

Listing 4.1: User Schema (backend/models/User.js)

Key design decisions:

- `select: false` on the password field prevents accidental exposure in API responses.
- The `pre("save")` hook guarantees the password is always hashed before storage.
- The `timestamps` option automatically adds `createdAt` and `updatedAt` fields.

4.3 RepairJob Model

The `RepairJob` schema is the core data entity of the application.

```
const RepairJobSchema = new mongoose.Schema({
  user:             { type: ObjectId, ref: "User", required: true },
  trackingCode:     { type: String, unique: true, uppercase: true },
  customerName:     { type: String, required: true },
  phoneNumber:      { type: String },
  carModel:         { type: String, required: true },
  licensePlate:     { type: String, uppercase: true },
  issueDescription: { type: String, required: true },
  status: {
    type: String,
    enum: ["pending","in-progress","completed","cancelled"],
    default: "pending",
  },
  estimatedCost: { type: Number, default: 0, min: 0 },
  actualCost:    { type: Number, default: 0, min: 0 },
  createdAt:     { type: Date, default: Date.now },
  completedAt:   { type: Date },
});
```

Listing 4.2: RepairJob Schema (backend/models/RepairJob.js)

4.3.1 Tracking Code Generation

Each repair job is automatically assigned a unique tracking code upon creation:

```
function generateTrackingCode() {
  const datePart   = new Date().toISOString().slice(2,10).replace(/-/g,"");
  const randomPart = Math.random().toString(36).slice(2,8).toUpperCase();
  return `RM-${datePart}-${randomPart}`;
```

```
  // Example: RM-250430-K3XP7A
}
```

Listing 4.3: Tracking Code Generator

The tracking code format `RM-YYMMDD-XXXXXX` encodes the creation date alongside a random 6-character alphanumeric suffix to ensure uniqueness. This code is shared with customers so they can track their vehicle without needing an account.

Field	**Type**	**Description**
user	ObjectId (ref)	Links job to its owner (garage user)
trackingCode	String	Auto-generated unique ID, e.g. RM-250430-K3XP7A
customerName	String	Vehicle owner name
carModel	String	Vehicle make/model
licensePlate	String	Vehicle registration plate
issueDescription	String	Problem description
status	Enum	pending / in-progress / completed / cancelled
estimatedCost	Number	Initial cost estimate (USD)
actualCost	Number	Final billed cost (USD)
completedAt	Date	Timestamp of completion

Table 4.1: RepairJob Fields

Chapter 5

API Design

5.1 RESTful Architecture

The backend exposes a RESTful JSON API organized under two route groups:

- `/api/auth` — Authentication (public)
- `/api/repairs` — Repair job management (protected, except customer tracking)

5.2 Authentication Endpoints

Method	**Endpoint**	**Description**
POST	/api/auth/signup	Create new user account
POST	/api/auth/login	Authenticate user, return JWT
GET	/api/auth/me	Get profile of the authenticated user

Table 5.1: Authentication Endpoints

5.3 Repair Job Endpoints

Method	Endpoint	Description	Auth
GET	/api/repairs/track/:code	Public customer tracking	None
GET	/api/repairs	List all jobs (with filter/search)	JWT
POST	/api/repairs	Create a new repair job	JWT
GET	/api/repairs/:id	Retrieve a single job	JWT
PUT	/api/repairs/:id	Update a job	JWT
DELETE	/api/repairs/:id	Delete a job	JWT
PATCH	/api/repairs/:id/complete	Mark job as completed	JWT

Table 5.2: Repair Job Endpoints

5.4 Query Filtering

The `GET /api/repairs` endpoint supports dynamic server-side filtering:

```
const { status, search } = req.query;
const filter = { user: req.user._id };

if (status && status !== "all") filter.status = status;

if (search) {
  filter.$or = [
    { customerName:  { $regex: search, $options: "i" } },
    { licensePlate:  { $regex: search, $options: "i" } },
    { carModel:      { $regex: search, $options: "i" } },
    { trackingCode:  { $regex: search, $options: "i" } },
  ];
}
const repairJobs = await RepairJob.find(filter).sort({
   createdAt: -1 });
```

Listing 5.1: Server-side Search and Filter Logic

Chapter 6

Frontend Architecture

6.1 Application Structure

The React frontend is a single-page application (SPA) built with **React 18**, styled with **Tailwind CSS**, and bundled with **Create React App**.

6.2 Core Components

Component	Responsibility
`App.jsx`	Root component; manages global state (auth, repairs, active tab)
`AuthForm.jsx`	Login and signup form with error display
`CustomerTrack.jsx`	Public tracking portal — searches by tracking code
`RepairForm.jsx`	Add/edit repair job form
`RepairList.jsx`	Renders the filtered list of repair cards
`RepairItem.jsx`	Single repair card with actions (edit, delete, complete)

Table 6.1: Frontend Component Roles

6.3 Navigation and Tabs

The application uses a **sidebar tab** navigation pattern. Five tabs are defined:

1. **Dashboard** — KPI cards, financial overview, recent repairs
2. **All Repairs** — Searchable, filterable repair job list

3. **New Repair** — Add or edit repair job form
4. **Completed** — Archive of finished jobs
5. **About** — Application info and API reference

6.4 State Management

All state is managed locally in `App.jsx` using React `useState` and `useMemo` hooks. There is no external state manager (Redux, Zustand). Key pieces of state include:

```
const [user,          setUser]          = useState(getStoredUser());
const [repairs,       setRepairs]       = useState([]);
const [editingRepair, setEditingRepair] = useState(null);
const [activeTab,     setActiveTab]     = useState("dashboard");
const [statusFilter,  setStatusFilter]  = useState("all");
const [searchTerm,    setSearchTerm]    = useState("");
```

Listing 6.1: Core State in App.jsx

6.5 Dashboard Metrics

The dashboard computes real-time statistics from the repairs array using `useMemo`:

```
const dashboard = useMemo(() => {
  const totalJobs      = repairs.length;
  const completedJobs  = repairs.filter(j => j.status === "completed").length;
  const pendingJobs    = repairs.filter(j => j.status === "pending").length;
  const inProgressJobs = repairs.filter(j => j.status === "in-progress").length;
  const totalEstimated = repairs.reduce((s,j) => s + Number(j.estimatedCost||0),0);
  const totalActual    = repairs.reduce((s,j) => s + Number(j.actualCost||0),0);
  const averageCost    = totalJobs ? totalEstimated / totalJobs : 0;
  return { totalJobs, completedJobs, pendingJobs,
           inProgressJobs, totalEstimated, totalActual, averageCost };
```

```
}, [repairs]);
```

Listing 6.2: Dashboard Computation

Chapter 7

Security Implementation

7.1 JWT Authentication

The application uses **JSON Web Tokens** (JWT) for stateless authentication.

1. On login/signup, the server signs a JWT with the user's `_id` and a secret key, setting an expiry of `30d`.
2. The token is returned to the client and stored in `localStorage`.
3. On every protected request, the client attaches the token as a `Bearer` header.
4. The `authMiddleware` verifies the token and injects the user into `req.user`.

```
const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization?.startsWith("Bearer")) {
    token = req.headers.authorization.split(" ")[1];
    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select("-password");
      next();
    } catch (err) {
      return res.status(401).json({ message: "Not authorized, token failed" });
    }
  } else {
    return res.status(401).json({ message: "Not authorized, no token" });
```

```
  }
};
```

Listing 7.1: Auth Middleware (middleware/authMiddleware.js)

7.2 Password Security

Passwords are hashed using **bcrypt** with a salt factor of 10 before being stored. Plaintext passwords are never persisted in the database.

7.3 CORS Policy

The server maintains an explicit allowlist of permitted client origins. Requests from unknown origins are rejected. Vercel preview deployments are handled by regex matching on the hostname pattern.

7.4 User Data Isolation

Every database query for repair jobs is filtered by `user: req.user._id`, guaranteeing that users can only read or modify their own records. Cross-user data access is structurally prevented.

Chapter 8

Deployment

8.1 Overview

The project is deployed to two cloud platforms:

Component	Platform	URL
Backend API	Render	https://mernproject-12kg.onrender.com
Frontend SPA	Vercel	https://mernproject-phi-eight.vercel.app
Database	MongoDB Atlas	Cloud-hosted, shared cluster

Table 8.1: Deployment Targets

8.2 Backend Deployment (Render)

```
Root Directory: backend
Build Command:  npm install
Start Command:  npm start

# Environment Variables:
MONGO_URI=mongodb+srv://<user>:<pass>@<cluster>/repairmanager
JWT_SECRET=<long_random_secret>
CLIENT_URLS=https://mernproject-phi-eight.vercel.app
```

Listing 8.1: Render Configuration

8.3 Frontend Deployment (Vercel)

```
Root Directory:   frontend
Build Command:    npm run build
Output Directory: build

# Environment Variable:
REACT_APP_API_URL=https://mernproject-12kg.onrender.com/api
```

Listing 8.2: Vercel Configuration

8.4 Health Check

The API exposes a `GET /health` endpoint that returns the current database connection status, enabling uptime monitoring.

Chapter 9

Features Summary

9.1 Implemented Features

Core Feature Set

- **Authentication:** User signup, login, JWT token issuance, protected routes
- **Repair Job CRUD:** Create, Read, Update, Delete repair records
- **Status Management:** pending → in-progress → completed / cancelled lifecycle
- **Tracking Codes:** Auto-generated unique codes (e.g., RM-250430-K3XP7A)
- **Customer Portal:** Public tracking page — no login required
- **Dashboard:** KPI cards, financial overview (estimated vs. actual revenue)
- **Search & Filter:** Real-time client-side search + server-side filtering by status
- **Responsive UI:** Collapsible sidebar, tab-based navigation, mobile-friendly
- **Cloud Deployment:** Render + Vercel + MongoDB Atlas

Chapter 10

Conclusion and Perspectives

10.1 Conclusion

This project successfully demonstrates a production-ready full-stack MERN web application. The **Repair Manager** system delivers a complete digital solution for auto repair shop management, covering the full lifecycle of a repair job from intake to completion.

The project gave the team hands-on experience with real-world software engineering concerns: REST API design, JWT security, NoSQL data modeling, React component architecture, cloud deployment, and environment configuration. The live deployment at `https://mernproject-phi-eight.vercel.app` validates the system end-to-end.

10.2 Future Enhancements

The following improvements are envisioned for future development iterations:

1. **Role-Based Access Control:** Differentiate between admin, mechanic, and receptionist roles.
2. **SMS/Email Notifications:** Alert customers when their vehicle status changes.
3. **Invoice Generation:** Automatically generate PDF invoices upon job completion.
4. **Analytics Charts:** Visual charts (line, bar, pie) for revenue and job trends.
5. **Mechanic Assignment:** Assign specific technicians to repair jobs.
6. **Image Upload:** Attach photos of vehicle damage to repair records.
7. **Mobile Application:** React Native port for garage staff on the shop floor.

10.3 Acknowledgements

The team expresses sincere gratitude to **Dr. HIOUANI Rima** for her guidance, feedback, and continuous support throughout the duration of this project.

Bibliography

[1] MongoDB, Inc. *MongoDB Documentation*. https://www.mongodb.com/docs/. Accessed 2025.

[2] OpenJS Foundation. *Express.js Documentation*. https://expressjs.com/. Accessed 2025.

[3] Meta Open Source. *React Documentation*. https://react.dev/. Accessed 2025.

[4] OpenJS Foundation. *Node.js Documentation*. https://nodejs.org/en/docs/. Accessed 2025.

[5] Mongoose. *Mongoose ODM Documentation*. https://mongoosejs.com/docs/. Accessed 2025.

[6] Auth0. *Introduction to JSON Web Tokens*. https://jwt.io/introduction. Accessed 2025.

[7] bcryptjs. *bcryptjs — npm*. https://www.npmjs.com/package/bcryptjs. Accessed 2025.

[8] Tailwind Labs. *Tailwind CSS Documentation*. https://tailwindcss.com/docs. Accessed 2025.

[9] Vercel Inc. *Vercel Deployment Documentation*. https://vercel.com/docs. Accessed 2025.

[10] Render. *Render Platform Documentation*. https://render.com/docs. Accessed 2025.